VHDL Assignment3

과목 디지털논리회로설계

학번 2023061549

이름 은지형

1. 설계 환경

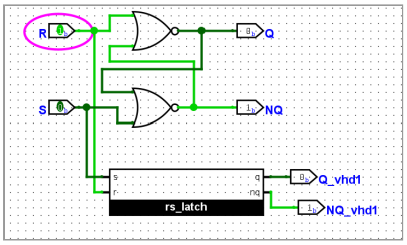
각 회로는 Logisim-evolution에서 기본 게이트만 사용하여 구현하였고(내장 Latch/Flip-Flop 미사용), 같은 동작을 VHDL로도 작성하였다. VHDL 코드는 Logisim의 VHDL Entity로 추가하여 Questa로 컴파일 및 시뮬레이션하였으며, 같은 입력에 대해 게이트 회로의 출력(Q, NQ)과 VHDL 출력(Q_vhd, NQ_vhd)을 비교하였다. 캡처 화면에서 밝은 초록색 선은 1, 어두운 초록색 선은 0이며, 분홍색 원으로 표시한 것은 해당 단계에서 변경한 입력이다.

2. RS Latch (NOR)

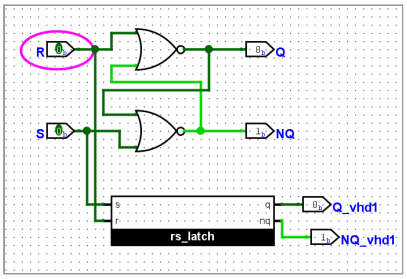
NOR 게이트 2개의 출력을 서로의 입력으로 연결하여 구성하였다. 시뮬레이션 시작 시 출력이 정해지지 않는 문제를 막기 위해 R 핀의 초기값을 1로 설정하였다.

[표 1] RS Latch 동작

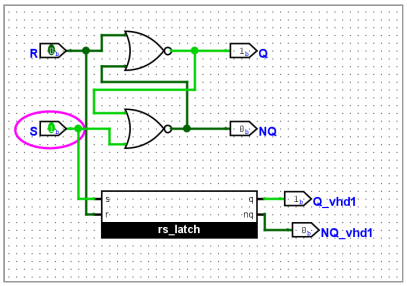
S	R	Q	동작
0	0	유지	Hold
1	0	1	Set
0	1	0	Reset
1	1	0 (Q=NQ=0)	Forbidden



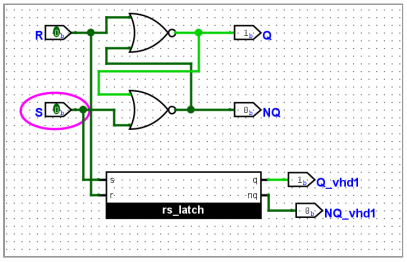
[그림 1] Reset (S=0, R=1) → Q=0



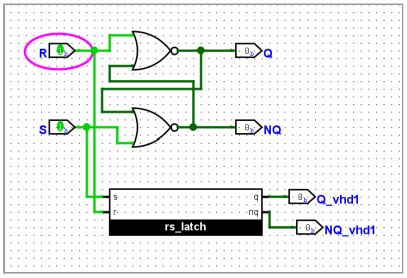
[그림 2] Hold (S=R=0) → Q=0 유지



[그림 3] Set (S=1, R=0) → Q=1



[그림 4] Hold (S=R=0) → Q=1 유지



[그림 5] Forbidden (S=R=1) → Q=NQ=0

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY rs_latch IS
  PORT (
    s  : IN  std_logic;
    r  : IN  std_logic;
    q  : OUT std_logic;
    nq : OUT std_logic
  );
END rs_latch;

ARCHITECTURE bhv OF rs_latch IS
  SIGNAL q_int  : std_logic := '0';
  SIGNAL nq_int : std_logic := '1';
BEGIN
  q_int  <= r NOR nq_int;
  nq_int <= s NOR q_int;

  q  <= q_int;
  nq <= nq_int;
END bhv;
```

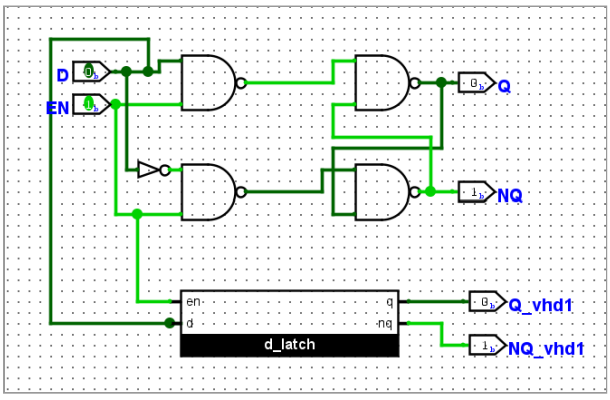
모든 입력 조합에서 게이트 회로와 VHDL의 출력이 같았다. S=R=1인 경우 Q와 NQ가 모두 0이 되어 서로 반대 값이라는 조건이 깨지므로 사용하지 않는 입력임을 확인하였다.

3. D Latch (NAND)

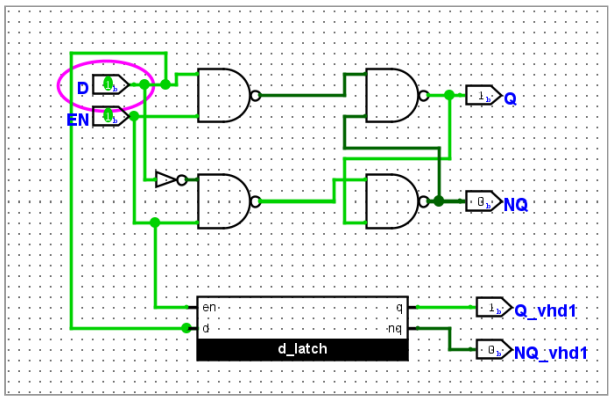
NAND 게이트 4개와 NOT 게이트 1개로 구성하였다. 앞단의 NAND 2개는 EN이 1일 때만 D 값을 통과시키고, 뒷단의 NAND 2개는 SR Latch로 동작하여 값을 저장한다. EN 핀의 초기값은 1로 설정하였다.

[표 2] D Latch 동작

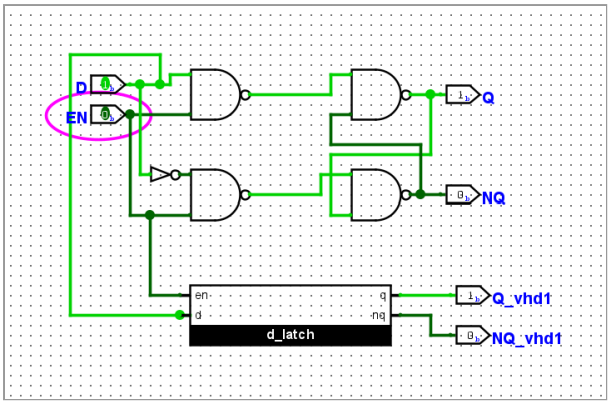
EN	D	Q	동작
1	0	0	D를 따라감
1	1	1	D를 따라감
0	X	유지	Hold



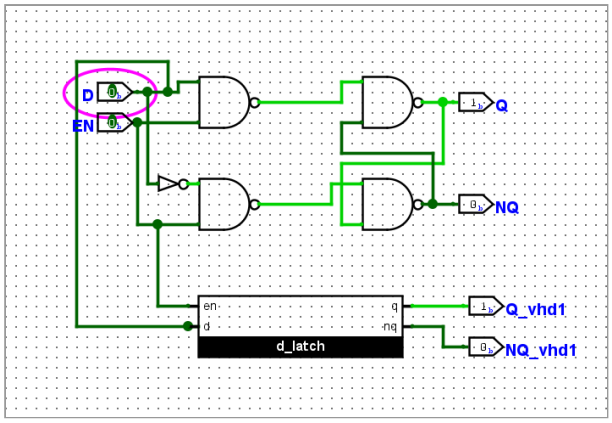
[그림 6] EN=1, D=0 → Q=0



[그림 7] EN=1, D=1 → Q=1



[그림 8] EN=0 → Q=1 유지



[그림 9] EN=0에서 D 변경 → Q 유지

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY d_latch IS
  PORT (
    en : IN  std_logic;
    d  : IN  std_logic;
    q  : OUT std_logic;
    nq : OUT std_logic
  );
END d_latch;

ARCHITECTURE bhv OF d_latch IS
  SIGNAL q_int : std_logic := '0';
BEGIN
  PROCESS(en, d)
  BEGIN
    IF (en = '1') THEN
      q_int <= d;
    END IF;
  END PROCESS;

  q  <= q_int;
  nq <= NOT q_int;
END bhv;
```

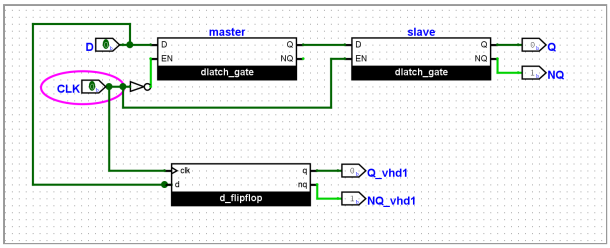
EN=1일 때는 Q가 D를 그대로 따라갔고, EN=0일 때는 D가 바뀌어도 Q가 유지되었다.

4. Positive-Edge D Flip-Flop (Master-Slave)

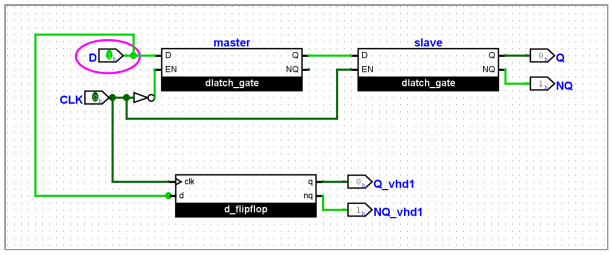
앞에서 만든 D Latch를 서브회로로 만들어 2개를 직렬로 연결하였다. Master의 EN에는 CLK를 NOT 게이트로 반전한 신호를, Slave의 EN에는 CLK를 그대로 연결하였다. CLK=0일 때 Master가 D 값을 받아두고, CLK가 0에서 1로 바뀌는 순간 Slave가 그 값을 출력하므로 상승 엣지에서만 Q가 바뀐다.

[표 3] D Flip-Flop 동작

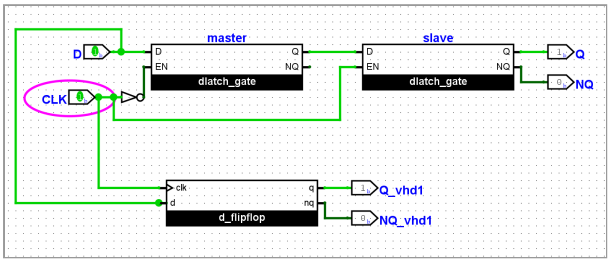
CLK	D	Q
↑ (0→1)	D	D
0, 1, ↓	X	유지



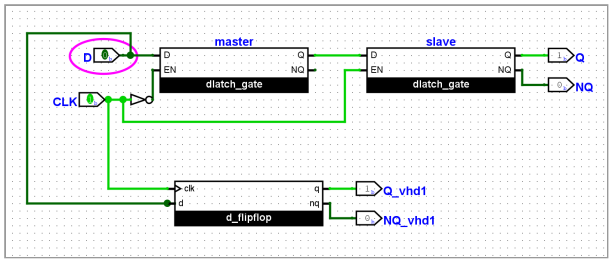
[그림 10] 초기 상태 (Q=0)



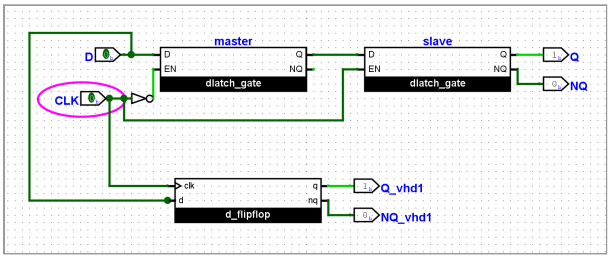
[그림 11] CLK=0에서 D=1 → Q 유지



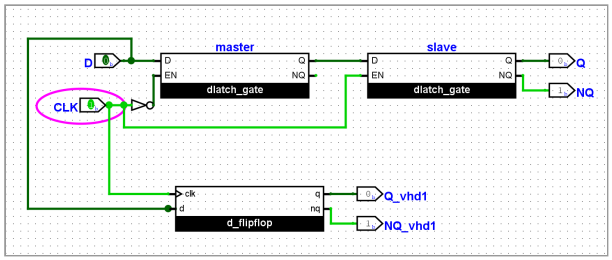
[그림 12] 상승 엣지, D=1 → Q=1



[그림 13] CLK=1에서 D 변경 → Q 유지



[그림 14] 하강 엣지 → Q 유지



[그림 15] 상승 엣지, D=0 → Q=0

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY d_flipflop IS
  PORT (
    clk : IN  std_logic;
    d   : IN  std_logic;
    q   : OUT std_logic;
    nq  : OUT std_logic
  );
END d_flipflop;

ARCHITECTURE bhv OF d_flipflop IS
  SIGNAL q_int : std_logic := '0';
BEGIN
  PROCESS(clk)
  BEGIN
    IF rising_edge(clk) THEN
      q_int <= d;
    END IF;
  END PROCESS;

  q  <= q_int;
  nq <= NOT q_int;
END bhv;
```

Q는 상승 엣지에서만 D 값을 반영하였고, CLK가 0 또는 1로 유지되는 동안이나 하강 엣지에서는 변하지 않았다.

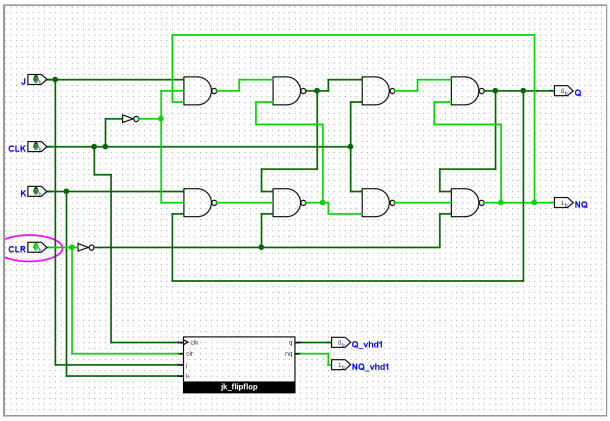
5. Positive-Edge JK Flip-Flop (NAND, Feedback)

강의자료의 NAND 4개로 된 JK 회로를 먼저 구성해 보았으나, 시뮬레이션 시작 시 출력이 정해지지 않은 상태(E)가 되었고 Q, NQ가 입력으로 되돌아가는 구조 때문에 J, K, CLK만으로는 이 상태가 풀리지 않았다. 또한 레벨 트리거 방식이어서 J=K=1이고 CLK=1인 동안 출력이 계속 바뀌는 현상이 생긴다.

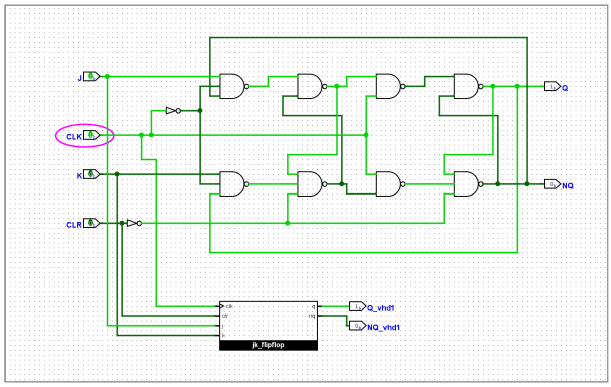
이를 해결하기 위해 조교님께서 공지로 안내해 주신 방법을 참고하여, 래치의 NAND 게이트를 3입력으로 바꾸고 CLR 입력을 추가해 초기값을 정할 수 있도록 하였다. 그리고 Toggle 동작을 확인할 수 있도록 Master-Slave 구조로 확장하였다(NAND 8개, NOT 2개). 입력단 NAND에 Q와 NQ를 되돌려 연결하여 J=K=1일 때 출력이 반전되도록 하였고, Master에 반전된 CLK를 넣어 상승 엣지에서 동작하도록 하였다. CLR은 VHDL 코드의 clr='1' 조건과 같은 의미가 되도록 NOT 게이트를 거쳐 연결하여, CLR=1일 때 Q=0이 되도록 하였다.

[표 4] JK Flip-Flop 동작

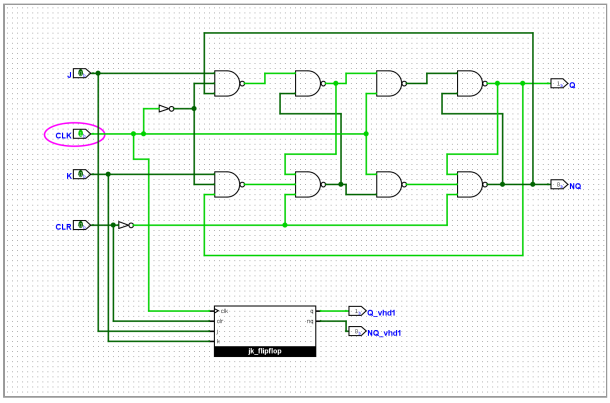
CLR	J	K	CLK	Q	동작
1	X	X	X	0	Clear
0	0	0	↑	유지	Hold
0	0	1	↑	0	Reset
0	1	0	↑	1	Set
0	1	1	↑	Q'	Toggle



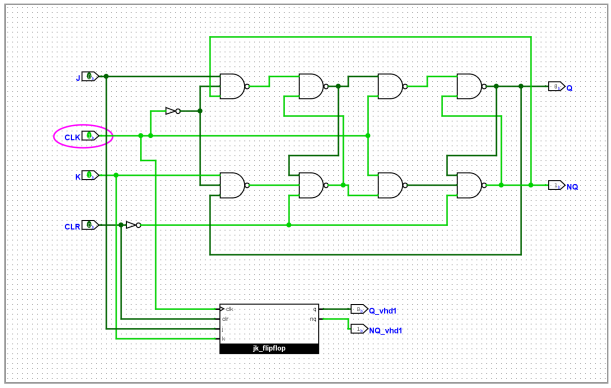
[그림 16] Clear (CLR=1) → Q=0



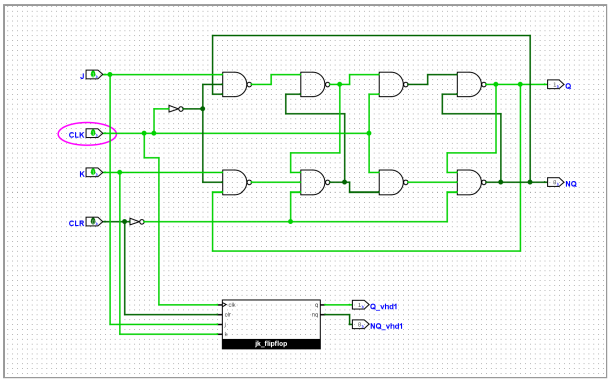
[그림 17] Set (J=1, K=0, ↑) → Q=1



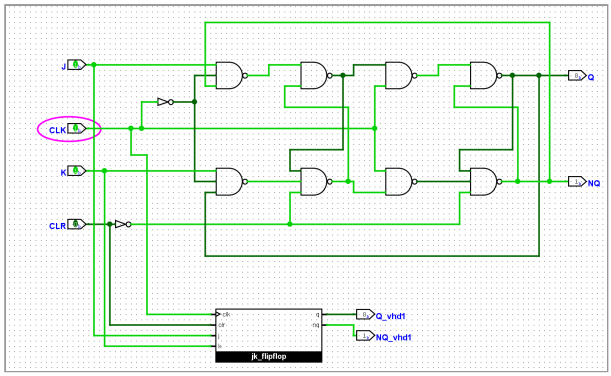
[그림 18] Hold (J=K=0, ↑) → Q 유지



[그림 19] Reset (J=0, K=1, ↑) → Q=0



[그림 20] Toggle (J=K=1, ↑) → Q=1



[그림 21] Toggle (J=K=1, ↑) → Q=0

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY jk_flipflop IS
  PORT (
    clk : IN  std_logic;
    clr : IN  std_logic;
    j   : IN  std_logic;
    k   : IN  std_logic;
    q   : OUT std_logic;
    nq  : OUT std_logic
  );
END jk_flipflop;

ARCHITECTURE bhv OF jk_flipflop IS
  SIGNAL q_internal : std_logic := '0';
BEGIN
  PROCESS(clk, clr)
  BEGIN
    IF (clr = '1') THEN
      q_internal <= '0';
    ELSIF rising_edge(clk) THEN
      IF (j = '0' AND k = '0') THEN
        q_internal <= q_internal;
      ELSIF (j = '0' AND k = '1') THEN
        q_internal <= '0';
      ELSIF (j = '1' AND k = '0') THEN
        q_internal <= '1';
      ELSIF (j = '1' AND k = '1') THEN
        q_internal <= NOT q_internal;
      END IF;
    END IF;
  END PROCESS;

  q  <= q_internal;
  nq <= NOT q_internal;
END bhv;
```

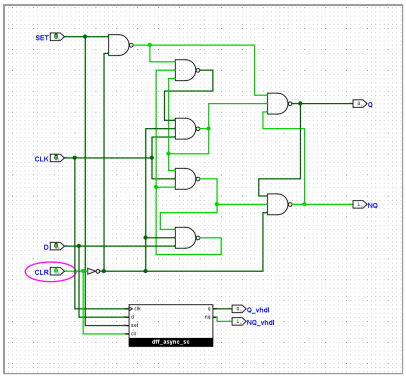
CLR로 초기화한 후 Hold, Set, Reset, Toggle이 모두 상승 엣지에서만 동작하였고, 게이트 회로와 VHDL의 출력이 일치하였다.

6. D Flip-Flop with Asynchronous Set and Clear

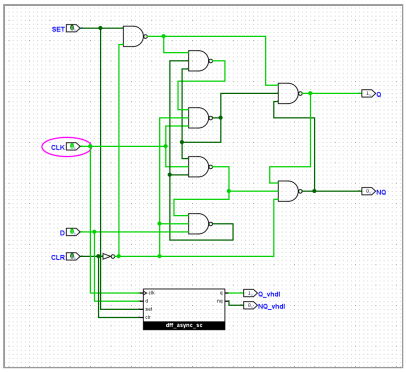
3입력 NAND 6개로 구성한 상승 엣지 D Flip-Flop에 SET과 CLR 입력을 추가하였다. CLR은 NOT 게이트를 거쳐 각 래치의 Clear 입력으로 연결하였고, SET은 NAND(SET, CLR')로 만든 신호를 Preset 입력으로 연결하였다. 이렇게 하면 CLR=1일 때 SET 신호가 무시되므로 Clear가 Set보다 우선한다. SET과 CLR은 CLK와 관계없이 바로 출력에 반영된다.

[표 5] 비동기 Set/Clear D Flip-Flop 동작

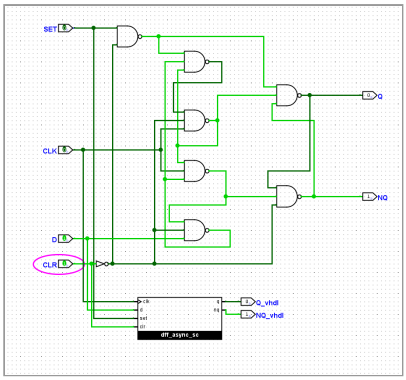
CLR	SET	CLK	Q	동작
1	0	X	0	Clear
1	1	X	0	Clear 우선
0	1	X	1	Set
0	0	↑	D	일반 D FF



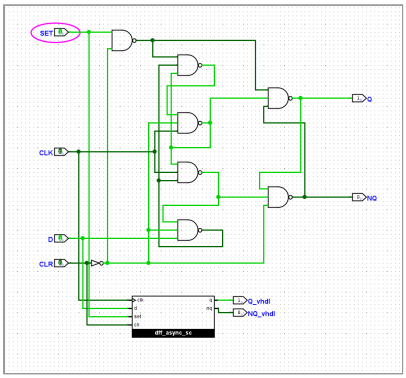
[그림 22] CLR=1로 초기화 → Q=0



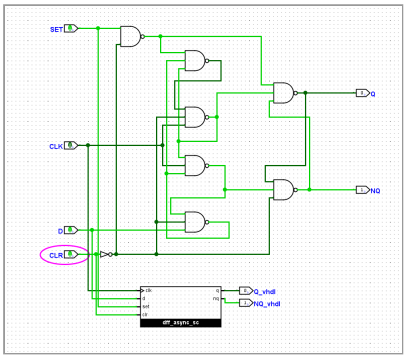
[그림 23] D=1, 상승 엣지 → Q=1



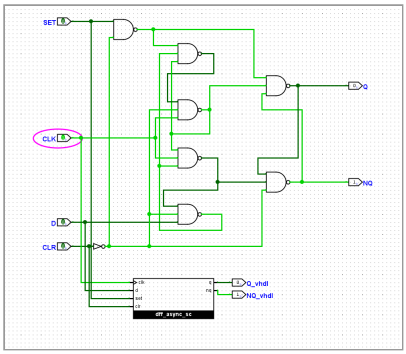
[그림 24] CLR=1 → 즉시 Q=0



[그림 25] SET=1 → 즉시 Q=1



[그림 26] SET=CLR=1 → Q=0



[그림 27] D=0, 상승 엣지 → Q=0

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dff_async_sc IS
  PORT (
    clk : IN  std_logic;
    d   : IN  std_logic;
    set : IN  std_logic;
    clr : IN  std_logic;
    q   : OUT std_logic;
    nq  : OUT std_logic
  );
END dff_async_sc;

ARCHITECTURE bhv OF dff_async_sc IS
  SIGNAL q_int : std_logic := '0';
BEGIN
  PROCESS(clk, clr, set)
  BEGIN
    IF (clr = '1') THEN
      q_int <= '0';
    ELSIF (set = '1') THEN
      q_int <= '1';
    ELSIF rising_edge(clk) THEN
      q_int <= d;
    END IF;
  END PROCESS;

  q  <= q_int;
  nq <= NOT q_int;
END bhv;
```

CLR과 SET은 클럭 없이 즉시 출력에 반영되었고, 두 입력이 모두 1일 때는 Clear가 우선하였다. 두 입력이 모두 0일 때는 상승 엣지에서 D 값을 저장하였다.

7. 결론

다섯 회로 모두 게이트 회로와 VHDL의 출력이 모든 테스트 입력에서 일치하였다. 피드백 구조를 가진 래치와 플립플롭은 시작 시 상태가 정해지지 않기 때문에 Reset이나 CLR 같은 초기화 입력이 필요하다는 것을 알 수 있었고, 두 개의 래치를 Master-Slave로 연결하면 엣지 트리거 동작을 만들 수 있음을 확인하였다.